

Proiect IDP - Milestone 1: The Imitation Game

Informații Generale

Membrii Echipei:

- Mihai Strejaru, Grupa 341C4

Asistent Responsabil: Alexandra Ion

Tema Proiectului: The Imitation Game - Platformă Multiplayer Web de Identificare AI în Timp Real

1. Descriere Tematică și Funcționalități

The Imitation Game este o platformă web de joc multiplayer competitiv unde utilizatorii intră în chat room-uri de 7 persoane pentru a identifica care dintre participanți este un agent AI.

Mecanica de Joc:

Jocul se desfășoară în runde succesive de 2 minute. În fiecare rundă:

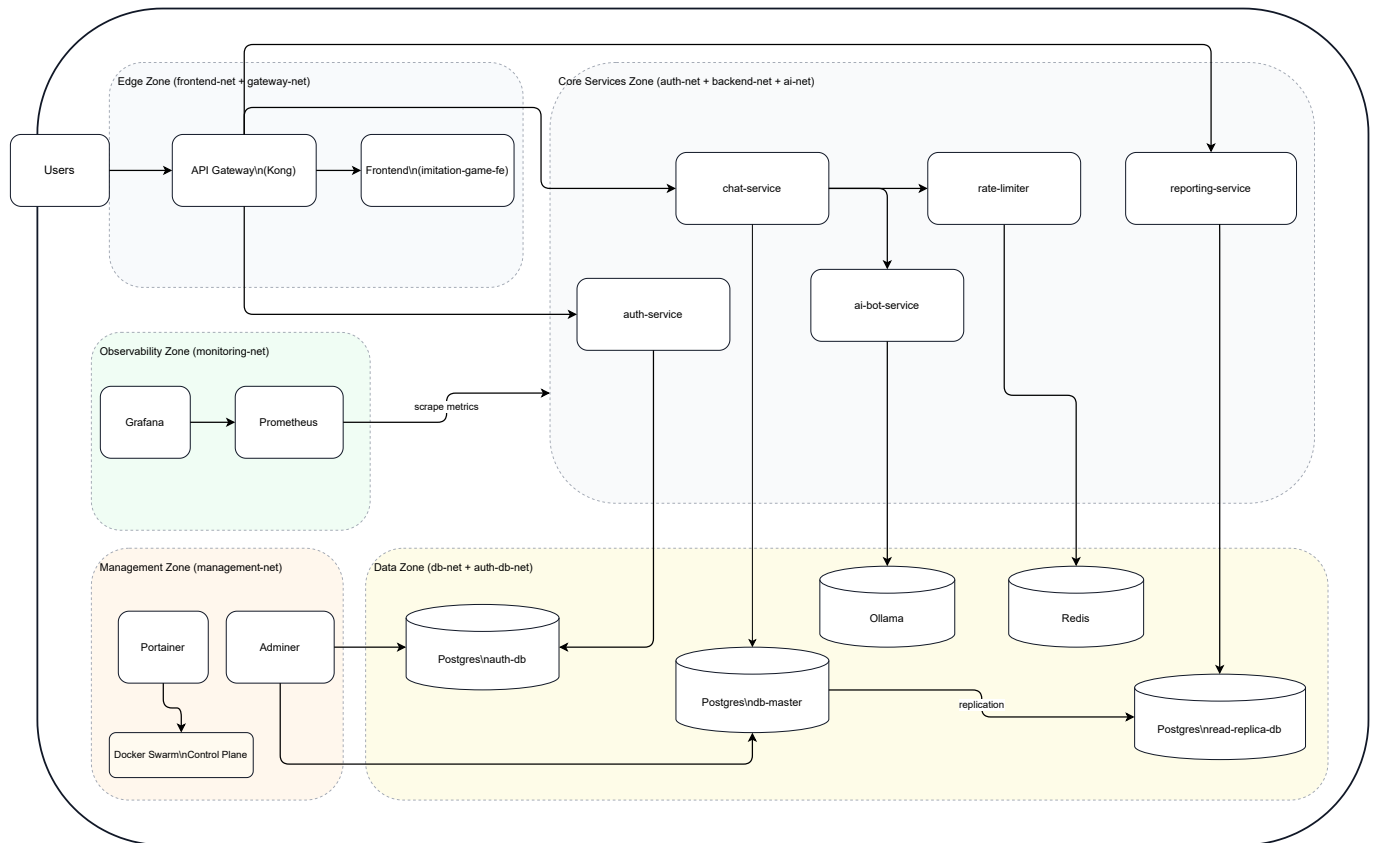
1. Toți 7 jucătorii sunt conectați printr-o interfață WebSocket și au o conversație în timp real
2. Unuia dintre jucători i se asignează secret rolul de "AI" (bot inteligent)
3. La finalul fiecărei runde, se lansează o **sesiune de vot** unde toți jucătorii votează pentru a elimina cine cred că este AI
4. Jucătorul cu cei mai mulți voturi este eliminat și devine spectator (nu mai poate trimite mesaje)
5. Jocul se termină când: (A) jucătorii umani identifică și elimină AI-ul → victorie oameni, sau (B) rămân doar 3 jucători → victorie AI

Funcționalități Principale:

- **Autentificare securizată** - POST /auth/register, POST /auth/login, POST /auth/refresh, GET /auth/userinfo, PUT /auth/logout, DELETE /auth/delete-account
- **Management game rooms** - GET /rooms, GET /rooms/{id}, POST /rooms/create, POST /rooms/join, POST /rooms/{id}/join, POST /rooms/{id}/leave, PUT /rooms/{id}, DELETE /rooms/{id}
- **Chat real-time WebSocket** - STOMP send pe /app/chat/{roomId} și subscribe pe /topic/messages/{roomId}
- **Mesaje persistente** - GET /rooms/{id}/messages, PUT /rooms/{roomId}/messages/{messageId}, DELETE /rooms/{roomId}/messages/{messageId}
- **Sistem inteligent de votare** - POST /rooms/{id}/vote, POST /rooms/{id}/voting/start, POST /rooms/{id}/voting/end, GET /rooms/{id}/results
- **Statistici personale** - GET /reports/me/stats, GET /reports/player/{orderId}/stats, GET /reports/player/username/{username}/stats
- **Clasamente globale** - GET /reports/leaderboard, GET /reports/global, GET /reports/games/recent, GET /reports/room/{roomId}/leaderboard, PUT /reports/settings, DELETE /reports/cache

- **Scalabilitate microservicii** - 3 replici chat-service, load balancing Kong, rate-limiting distribuit pe DELETE/PUT/POST
- **Monitorizare real-time** - Dashboard Grafana, metrice Prometheus ([/actuator/prometheus](#)), logs Portainer

2. Diagramă Arhitectură



3. Descriere Componente și Tehnologii

Frontend (React + Vite + TypeScript): aplicație SPA cu ecrane pentru autentificare, lobby, cameră de joc, votare, rezultate și statistici. Comunică prin HTTP cu backend-ul și prin WebSocket/STOMP pentru chat în timp real.

Auth-Service (Python FastAPI): gestionează identitatea utilizatorilor (înregistrare, login, refresh, userinfo, logout, ștergere cont) și emite token-uri JWT validate de celelalte servicii. Persistă datele în PostgreSQL ([auth-db](#)).

Chat-Service (Spring Boot, replicat): serviciul central al jocului. Expune API-uri pentru camere (creare, join/leave, start, update, delete), mesaje și votare; rulează și endpoint-uri WebSocket pentru mesaje live. Datele de joc sunt păstrate în PostgreSQL ([db-master](#)).

AI-Bot-Service (Spring Boot): generează răspunsuri pentru participantul AI pe baza contextului din cameră. Apelează modelul LLM prin Ollama și trimite rezultatele în fluxul de chat.

Rate-Limiter (Spring Boot + Redis): limitează numărul de cereri pe utilizator, pentru protecție anti-spam/DoS. Oferă endpoint-uri de verificare, status și reset.

Reporting-Service (Spring Boot): calculează statistici personale și globale (leaderboard, istoric jocuri, indicatori de performanță). Pentru a nu încărca fluxul de scriere, citește din replica read-only (**read-replica-db**).

Kong API Gateway: punct unic de intrare în sistem, cu rutare către frontend și microservicii (**/auth**, **/api**, **/ws**, **/reports**), plus load-balancing pentru replicile **chat-service**.

Persistență și conectivitate: PostgreSQL master + read replica (WAL streaming replication), Redis pentru cache/counters/pub-sub, rețele Docker pentru comunicarea inter-servicii.

Observabilitate și operare: Prometheus colectează metrice, Grafana oferă dashboard-uri, iar Portainer este folosit pentru administrarea stack-ului Docker Swarm.

4. Responsabilități Membrii Echipei

Mihai Strejaru - Solo Development (toate componentele)

Infrastructure & Deployment:

- Docker Swarm stack design (stack.yml v3.9)
- 9 rețele overlay, volumes persistent, constraints placement
- Kong declarative config (kong.yml)
- Prometheus + Grafana setup cu dashboards
- Portainer integration
- CI/CD pipeline (GitHub Actions)

Backend Services:

- **Auth-Service (FastAPI):** register, login, refresh, userinfo, logout, delete-account; auth-db schema; JWT tokens
- **Chat-Service (Spring Boot):** game logic (create room, join, vote), WebSocket upgrade, Redis pub/sub, db-master queries, rate-limit integration
- **Rate-Limiter:** atomic increments Redis counters, check endpoint
- **Reporting-Service:** leaderboard queries pe read-replica-db, player stats aggregation
- **AI-Bot-Service:** Ollama integration, prompt engineering, inference calls

Frontend:

- **React/Vite SPA:** Login form, Lobby (room list), ChatRoom (WebSocket 7-player), Voting UI, Results screen, Leaderboard, Stats pages
- Navigation, state management, WebSocket client

Databases:

- PostgreSQL schema: users, game_rooms, room_players, message_logs, vote_logs, user_profiles
- Replication setup: WAL streaming, pg_basebackup, hot-standby config

Testing & Documentation:

- Unit tests servicii Spring Boot

- Integration tests Chat-Service ↔ DB
 - End-to-end WebSocket tests
 - API documentation (Swagger/OpenAPI)
 - Architecture documentation
-

5. Repository GitHub

Proiectul este implementat într-un **monorepo** centralizat cu structura pe directoare per microserviciu:

- **GitHub:** <https://github.com/Cristiansen13/The-Imitation-Game>
 - Conține: imitation-game-fe/, imitation-game/auth-service/, imitation-game/chat-service/, imitation-game/rate-limiter/, imitation-game/reporting-service/, imitation-game/ai-bot-service/, imitation-game/docker/, documentation/